

6주차 보충 · Simulink 속성 입문 (2회 6시간)

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-03

- **과목:** 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- **이번 주차 학습 내용:** Simulink 를 직접 만들어 본다. 열두 개의 빈칸 모델을 채운다
- **구성:** 1일차 AF (문법 3시간) → 2일차 GL (7~9주차에서 쓰는 블록 3시간)

★ 시작 전 확인

- MATLAB R2024b + Simulink 가 설치되어 있을 것
- MATLAB 은 어느 정도 안다고 전제한다 — 변수, **for**, **if**, 함수 정의
- Simulink 는 전혀 몰라도 된다. 처음 접하는 경우이라고 가정하고 쓴 문서다

i 이번 주차에는 ROS 도 Gazebo 도 VRX 도 쓰지 않는다

- 인터넷도, WSL 도, 배도 필요 없다. MATLAB 하나만 켜면 된다
- 이번 주차의 학습 대상은 Simulink 라는 도구의 사용법뿐이다
- 동역학, 제어 이론, 배의 물리는 하나도 안 나온다. 그건 7·8주차에 한다
- 예제 숫자는 전부 아무 의미 없는 숫자다. 의미를 찾지 말 것

★ 두 번에 나눠 한다

	절	무엇을
1일차 (3시간)	A~F	Simulink 문법 — 블록·서브시스템·버스·PID·로깅
2일차 (3시간)	G~L	7~9주차 모델에 실제로 들어 있는 블록들

2일차는 "쓸모 있는 블록 모음"이 아니라 본 과목의 모델을 읽기 위한 어휘다.

Unit Delay 를 모르면 9주차 대수 루프를 이해할 수 없고,

Transfer Fcn 을 모르면 7주차 모터 모델을 읽을 수 없다.

구성은 MathWorks 공식 교재 [Simulink Fundamentals.pdf](#) 의 장 순서를 따랐다.

이 주차를 마치면 할 수 있어야 하는 것

1. 빈 모델에 블록을 놓고 선으로 이어 실행하기
2. 솔버를 고정 스텝 0.05 초로 설정하기
3. MATLAB Function 블록에 코드를 써서 원하는 계산 넣기
4. 여러 블록을 Subsystem 한 칸으로 묶고, Goto/From 으로 선 없이 잇기
5. 버스를 만들고, 원소를 꺼내고, 한 칸만 바꿔치기
6. PID 블록을 놓고 포화·안티와인드업 켜기
7. 블록 값에 변수 이름을 쓰고, 신호를 로깅해서 MATLAB 으로 꺼내기
8. Mux·Demux·Selector 로 신호를 묶고 풀기
9. Unit Delay 로 누적기를 만들고, 대수 루프를 끊는 법 설명하기
10. Integrator·Transfer Fcn 으로 미분방정식과 1차 지연 만들기
11. Switch·Multiport Switch·Stop Simulation 으로 모드 전환과 종료 만들기
12. Enabled·Triggered 서브시스템의 차이를 실측으로 설명하기
13. 서브시스템에 마스크를 씌워 파라미터 다이얼로그 만들기

14. 모델을 코드로 생성하는 방식이 왜 필요한지 설명하기

준비물

항목	내용
소프트웨어	MATLAB R2024b + Simulink
필요 없는 것	ROS 2, WSL, Gazebo, VRX, 인터넷
배포 파일	<code>W06_0_simulink/</code> 폴더 (모델 24개 + 생성 스크립트 2개)
소요 시간	3시간 x 2회 (1일차 AF, 2일차 GL)

1부 · 이론 (15분)

1-1. Simulink 는 무엇이 다른가

- MATLAB 스크립트: 내가 순서를 정해 한 줄씩 실행
- Simulink: 시간이 저절로 흐르고, 매 순간 모든 블록이 한 번씩 계산됨

	MATLAB 스크립트	Simulink
실행 단위	한 줄	한 시간 스텝
시간	내가 <code>for</code> 로 만들	도구가 알아서 흘러보냄
표현	글자	블록과 선
잘 맞는 일	계산, 분석, 그래프	되먹임 루프, 실시간 제어

- 본 과목에서 Simulink 를 쓰는 이유는 되먹임 루프 때문이다
- 출력이 다시 입력으로 돌아오는 구조를 글자로 쓰면 금방 엉킨다

1-2. 화면 구성 — 네 군데만 알면 된다

Simulink 편집기 — 이 네 군데만 알면 된다

① 툴스트립

▶ Run 정지 시간 20 Model Settings

② 라이브러리

Sources
Sinks
Math Operations
Signal Routing
Discrete
Discontinuities
User-Defined
Ports & Subsystems
여기서 끌어다 놓는다

③ 캔버스 — 블록을 놓고 선으로 잇는 곳

Sine

Gain

Scope

블록 = 계산 한 덩어리 / 선 = 흘러가는 값

④ Ctrl + E

솔버 설정

이 수업 설정

고정 스텝
이산(discrete)
0.05 초

MATLAB 스크립트와 다른 점 — 시간이 저절로 흐른다. 0.05초마다 모든 블록이 한 번씩 계산된다

- 그림 읽는 법

번호	이름	하는 일	여는 법
①	툴스트립	실행, 정지 시간 입력	항상 위에 있음
②	라이브러리 브라우저	블록을 골라 끌어다 놓음	Ctrl + Shift + L
③	캔버스	블록을 놓고 선으로 잇는 곳	모델 창 가운데
④	Model Settings	솔버 설정	Ctrl + E

1-3. 용어 여섯 개

용어	뜻	비유
블록	계산 한 덩어리	함수 하나
포트	블록의 입출력 구멍	함수의 인자와 반환값
신호선	블록 사이로 값이 흐르는 선	변수 하나
서브시스템	여러 블록을 담은 한 칸	함수를 파일로 뺀 것
버스	여러 신호를 한 줄로 묶은 것	구조체
솔버	시간을 얼마씩 진행할지 정하는 규칙	<code>for</code> 문의 스텝

1-4. 솔버 — 본 과목에서는 한 가지만 사용한다

★ 고정 스텝 · 이산 · 0.05 초

이 값을 모든 모델에 똑같이 쓴다. 다른 설정은 이번 주차 다루지 않는다.

- 고정 스텝(Fixed-step) — 항상 0.05 초씩 일정하게 진행
- 이산(discrete) — 미분방정식을 풀지 않음. 본 과목 모델에 연속 상태가 없음
- 0.05 초 = 20 Hz — 1초에 20번 계산
- 가변 스텝(Variable-step)을 쓰면 스텝 간격이 들쭉날쭉해진다
- 나중에 실제 장비와 주기를 맞출 수 없게 된다

1-5. 이번 주차에 학습하는 블록

오늘 배우는 블록 — 이것이 전부다

Simulink 블록은 수백 개다. 그중 실제로 쓰는 것만 골랐다

A · 신호를 만들고 본다

Constant — 고정된 값
Step — 어느 시각에 뛰는 값
Sine Wave — 사인파
Digital Clock — 지금 시각
Scope / Display — 눈으로 확인

B · 계산한다

Gain — 곱하기
Sum — 더하기 / 빼기
Product — 곱하기 / 나누기
Saturation — 위아래로 자르기
MATLAB Function — 코드로

C · 선을 정리한다

Subsystem — 여러 블록을 한 칸에
In1 / Out1 — 그 칸의 입출력
Goto / From — 선 없이 잇기
Bus Creator — 여러 개를 한 줄로
Bus Selector / Assignment

D · 시간을 기억한다

Unit Delay — 한 스텝 전 값
Discrete-Time Integrator
Discrete Transfer Fcn

E · 제어한다

Discrete PID Controller
— P / I / D 를 한 블록으로
— 안티와인드업 옵션 포함

F · 결과를 꺼낸다

To Workspace — 변수로 내보내기
신호 로깅 — 선에 이름 붙이기
Data Inspector — 겹쳐 보기

Stateflow 차트는 오늘 다루지 않는다. 미션 상태기계는 13주차에 기존 모델로 배운다
ROS 2 블록(Subscribe / Publish)도 오늘은 없다. 6주차에 이 블록들 위에 얹는다

- Simulink 블록은 수백 개다. 그중 **실제로 쓰는 것만** 골랐다
- 고른 기준: 본 과목의 기존 모델들이 실제로 사용하는 블록
- **Stateflow 차트는 이번 주차 없다** — 13주차에 기존 모델로 배운다
- **ROS 2 블록도 이번 주차 없다** — 6주차에 이번 주차에 학습한 내용 위에 얹는다

2부 · 실습

★ 실습 파일은 두 개씩 짝이다

- `..._todo.slx` — **빈칸본**. 학습자가 직접 작성하는 파일
- `..._done.slx` — **완성본**. 막혔을 때 열어서 대조하는 것

먼저 `_todo` 를 혼자 해 보고, 다 됐거나 막혔을 때 `_done` 을 연다.
각 모델의 캔버스 아래쪽에 **할 일이 적힌 메모**가 붙어 있다.

시작하기

```
cd('<배포 폴더>/W06_0_simulink')
```

- 모델을 여는 방법은 두 가지

```
open_system('SB1_first_todo')
```

- 또는 MATLAB 파일 탐색기에서 `.slx` 파일을 더블클릭

1일차 · Simulink 문법

1일차는 A~F 절이다

Simulink 라는 도구의 사용법만 익힌다. 배도 제어도 나오지 않는다.

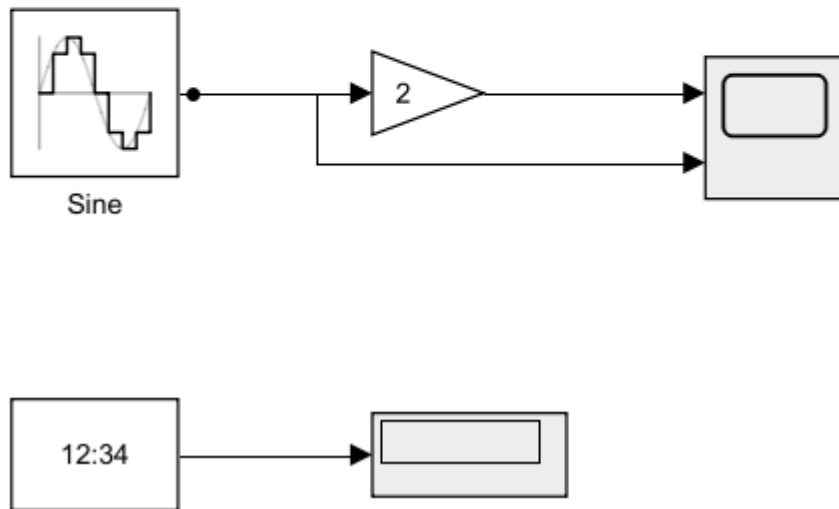
A. 첫 모델과 솔버 (20분)

[A] 첫 모델

Sine -> Gain(2) -> Scope. 원래 신호도 Scope 2번 포트에 함께 넣었다.

Digital Clock 은 지금 시각을 내보낸다. Display 로 확인.

솔버: 고정 스텝 / 이산 / 0.05 s, 정지 시간 20 s.



1 블록 색은 역할 표시다

이 과목의 모든 Simulink 모델은 같은 색 규칙을 따른다.

연보라 = ROS 통신, 주황 = 계산·제어, 회색 = 관찰(Scope·로깅), 흰색 = 설정값(Constant).

7주차부터는 파랑(유도)·노랑(추진기)·초록(운동모델)이 더해진다.

배치를 흐트러뜨렸으면 `tidy_layout('모델이름')` 한 줄로 되돌린다.

A-1. 모델 열기

```
open_system('SB1_first_todo')
```

- Sine 과 Clock 두 블록만 놓여 있다. 나머지는 학습자가 놓는다

A-2. 블록 놓기

1. 툴스트립에서 Library Browser 클릭 (또는 Ctrl + Shift + L)
2. 왼쪽 목록에서 Math Operations 클릭
3. Gain 블록을 찾아 캔버스로 끌어다 놓는다
4. 같은 방법으로 Sinks → Scope 를 놓는다

🔧 블록 배치를 빠르게 하는 방법

캔버스 빈 곳을 **더블클릭**하고 블록 이름을 타이핑하면 바로 찾아진다.

gain 이라고 치면 Gain 블록이 나온다.

A-3. 선으로 잇기

- 블록의 **오른쪽 화살표(출력 포트)** 에서 마우스를 누른 채
- 다음 블록의 **왼쪽 화살표(입력 포트)** 까지 끌면 선이 그려진다

하고 싶은 것	방법
블록 잇기	출력 포트에서 입력 포트로 드래그
블록 복사	Ctrl 누른 채 블록을 드래그
이름 바꾸기	블록 이름을 클릭하고 타이핑
선 지우기	선을 클릭하고 Delete
한 신호를 여러 곳으로	기존 선에서 Ctrl 누른 채 드래그

A-4. Gain 값 바꾸기

- Gain** 블록을 **더블클릭** → 값을 **2** 로 → **OK**

A-5. Scope 입력 포트 늘리기

- Scope** 를 **더블클릭**해 창을 연다
- 창 안 툴바의 **톱니바퀴(Configuration Properties)** → **Number of input ports** 를 **2** 로
- Sine** 의 출력에서 **Ctrl** 드래그로 선을 하나 더 빼서 **Scope** 2번 포트에 잇는다

A-6. 솔버 설정 — 핵심 항목

⚠ **빈칸본은 의도적으로 가변 스텝으로 되어 있다**

이걸 고치지 않으면 나중에 모든 모델이 이상하게 돈다.

- Ctrl** + **E** (또는 툴스트립 **Model Settings**)
- 왼쪽에서 **Solver** 선택
- 아래처럼 맞춘다

항목	값
Type	Fixed-step
Solver	discrete (no continuous states)
Fixed-step size	0.05
Stop time	20

- OK**

A-7. 실행

- 툴스트립의 ▶ **Run** 을 누른다
- Scope** 를 **더블클릭**하면 사인파 두 개가 보인다 (원래 신호, 2배 신호)
- Display** 에는 시각이 흘러가는 것이 보인다

A-8. 샘플 타임 색으로 확인하기

- 툴스트립 **Debug** → **Information Overlays** → **Sample Time** → **Colors**
- 모든 블록과 선이 **같은 색**이면 0.05 초가 잘 퍼진 것이다

🔍 색이 다른 블록은 샘플 주기가 다르다

나중에 원인 모를 문제가 생기면 이 화면부터 켜 볼 것.

확인

- ☐ Scope 에 사인파 두 개가 보인다
 - ☐ **Ctrl** + **E** 에서 Fixed-step / discrete / 0.05 / 20 으로 되어 있다
-

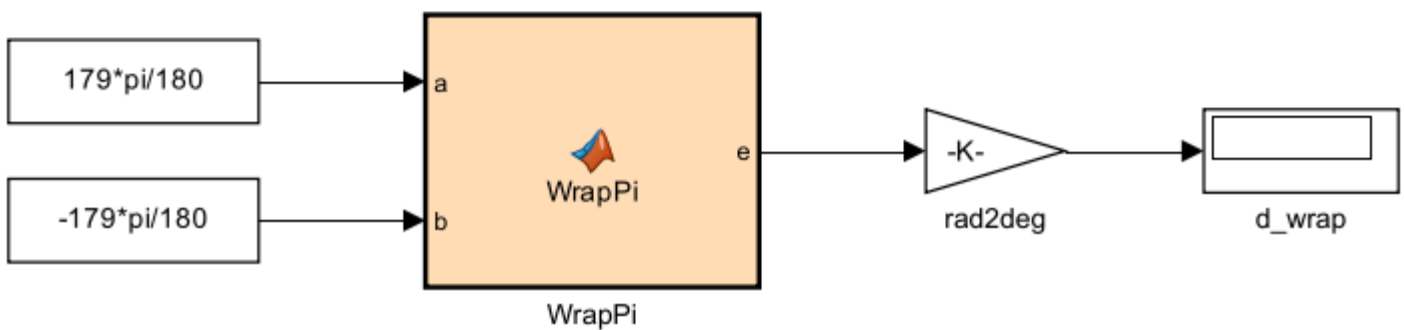
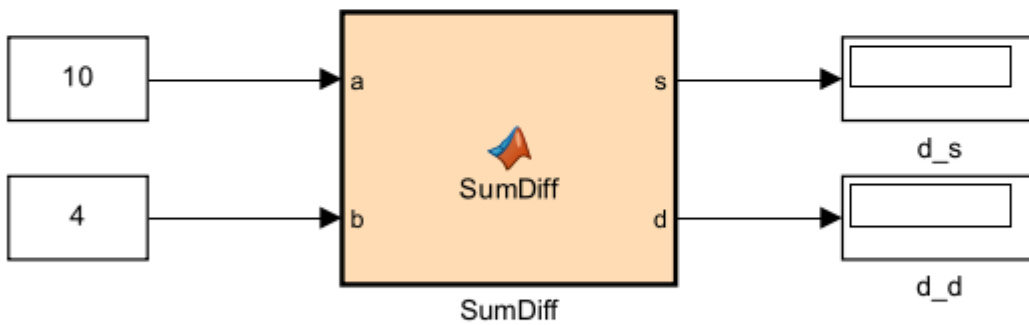
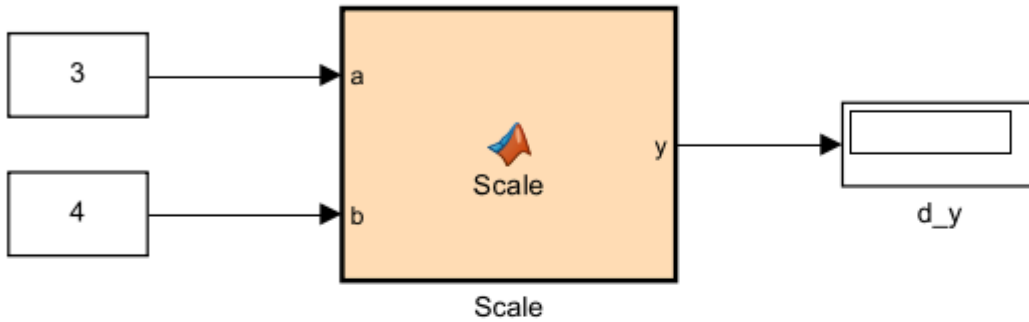
B. MATLAB Function 블록 (35분)

[B] MATLAB Function

함수의 첫 줄을 고치면 블록의 포트가 저절로 바뀐다. 이것만 기억하면 된다.

WrapPi의 Display에 -2가 떠야 정답이다.

358도가 아니라 -2도다. 반대쪽으로 2도만 돌면 된다는 뜻이다.



★ 이번 주차 가장 중요한 절이다

본 과목의 기존 모델에서 **가장 많이 쓰이는 블록**이 이것이다.

계산이 조금만 복잡해져도 블록을 수십 개 그리는 것보다 **코드 세 줄**이 낫다.

```
open_system('SB2_mfcn_todo')
```

B-1. 기억할 것은 한 가지뿐

★ 함수의 첫 줄을 고치면 블록의 포트가 저절로 바뀐다

```
function y = Scale(a)          % 입력 1개, 출력 1개
function y = Scale(a, b)      % 입력 2개, 출력 1개 <- 포트가 하나 늘어남
function [s, d] = SumDiff(a, b) % 입력 2개, 출력 2개
```

- 포트를 마우스로 추가하는 것이 아니다. 코드가 포트를 만든다

B-2. 첫 번째 함수 고치기

1. **Scale** 블록을 더블클릭 → 코드 편집기가 열린다
2. 첫 줄을 이렇게 고친다

```
function y = Scale(a, b)
%#codegen
y = a * b;
```

3. **Ctrl** + **S** 로 저장하고 편집기를 닫는다
4. 블록에 입력 포트가 2개가 된 것을 확인한다
5. 연결되지 않고 남아 있던 **b1** 상수를 2번 포트에 잇는다
6. **Run** → **Display** 에 **12** 가 뜨면 성공 (3×4)

i **%#codegen** 은 무슨 뜻인가

"이 코드는 C 코드로 바꿀 수 있게 써 두었다"는 표시다.
지금은 없어도 돌아가지만, 습관적으로 붙여 둔다.

B-3. 출력이 두 개인 함수

- **SumDiff** 블록을 더블클릭해서 고친다

```
function [s, d] = SumDiff(a, b)
%#codegen
s = a + b;
d = a - b;
```

- **b2** 를 2번 입력에 잇고, 늘어난 2번 출력에 **Display** 를 하나 더 놓아 잇는다
- **Run** → **14** 와 **6** 이 뜨면 성공

B-4. 각도 wrap 함수

- 이건 실제로 계속 쓰게 될 계산이다. 순수 수학이고 배와는 상관없다

```
function e = WrapPi(a, b)
%#codegen
d = a - b;
e = atan2(sin(d), cos(d));
```

- **ang_b** 를 2번 입력에 잇는다
- **Run** → **Display** 에 **-2** 가 뜨면 성공

★ 왜 -2 인가

- 입력은 179도 와 -179도 다
- 단순히 빼면 $179 - (-179) = 358$ 도
- 그런데 실제로는 반대쪽으로 2도만 가면 된다
- `atan2(sin, cos)` 가 이 "가까운 쪽"을 골라 준다. 그래서 -2
- 358 이 났다면 wrap 이 빠진 것이다

B-5. 일부러 오류를 내 본다

★ 오류 메시지를 읽는 연습이 실력이다

아래를 일부러 만들어 보고, 어떤 메시지가 나오는지 눈에 익혀 둘 것.

① 분기에서 출력을 안 채우면

```
function y = Scale(a, b)
%#codegen
if a > 0
    y = a * b;
end % a <= 0 일 때 y 가 없다
```

- 나오는 메시지: `Output argument 'y' is not assigned on some execution paths`
- 해결: `else` 를 넣어 모든 경우에 `y` 를 정한다

② 배열 크기를 키우면

```
function y = Scale(a, b)
%#codegen
v = [];
for k = 1:3
    v = [v, a*k]; % 크기가 계속 변한다
end
y = v(1) + b;
```

- Simulink 는 크기가 변하지 않는 배열을 좋아한다
- 해결: `v = zeros(1,3);` 로 미리 자리를 잡아 둔다

🔪 지원되지 않는 MATLAB 함수가 있다

`input`, `disp`, `figure` 같은 것은 MATLAB Function 블록 안에서 쓸 수 없다.
계산만 하는 코드를 쓴다고 생각하면 된다.

확인

- ☐ `Scale` 에 12 가 뜬다
- ☐ `SumDiff` 에 14, 6 이 뜬다
- ☐ `WrapPi` 에 -2 가 뜬다
- ☐ 출력 미할당 오류 메시지를 직접 봤다

C. Subsystem 과 Goto/From (30분)

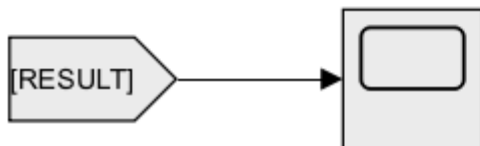
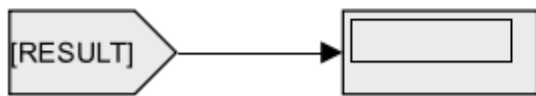
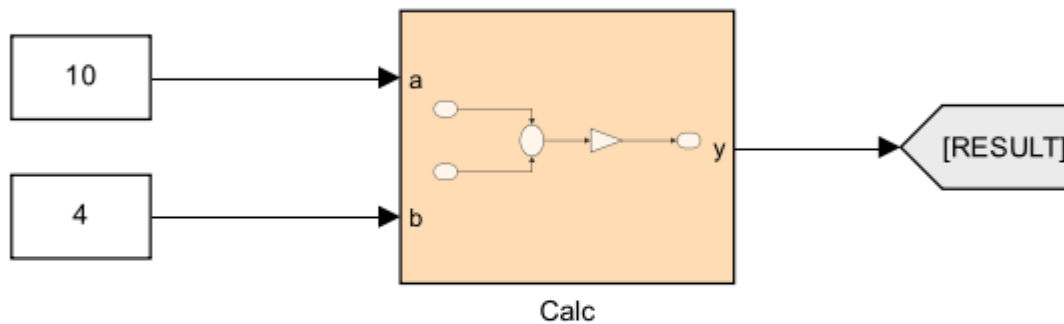
[C] Subsystem 과 Goto/From

Calc 를 더블클릭하면 안으로 들어간다. 안에는 Sum 과 Gain 뿐이다.

포트 이름 a, b, y 는 안쪽 In1/Out1 블록의 이름이 그대로 나온 것이다.

Goto[RESULT] 하나에 From[RESULT] 두 개가 붙어 있다. 선을 끌지 않았다.

정답: $(10 - 4) * 2 = 12$



```
open_system('SB3_subsys_todo')
```

- 지금은 블록이 전부 한 화면에 평평하게 놓여 있다
- 블록이 20개만 넘어가도 이 상태로는 못 본다

C-1. Subsystem 으로 묶기

1. **Sum** 과 **K** 두 블록만 마우스로 드래그해서 선택한다
 2. **Ctrl** + **G** 를 누른다
 3. 두 블록이 한 칸으로 접힌다
- 새 칸의 이름을 클릭해서 **Calc** 로 바꾼다
 - **Calc** 를 더블클릭하면 안으로 들어간다. 아까 그 두 블록이 있다
 - 위로 나오려면 톨스트립 왼쪽 위의 ↑ 화살표를 누른다

C-2. 포트 이름 바꾸기

- **Calc** 안으로 들어가면 **In1**, **In2**, **Out1** 블록이 생겨 있다
- 이 블록들의 이름을 바꾸면 바깥에서 보이는 포트 이름이 따라 바뀐다

안쪽 블록	바깥 이름
In1	a
In2	b
Out1	y

- 위로 나와서 **Calc** 칸을 보면 포트에 **a**, **b**, **y** 라고 적혀 있다

i 이것이 서브시스템의 요점이다

바깥에서는 " **a** 와 **b** 를 넣으면 **y** 가 나온다"만 알면 된다.
안에서 무엇을 하는지는 필요할 때만 열어 본다. 함수와 똑같다.

C-3. Goto / From 으로 선 없애기

- Calc** 에서 **Display** 로 가는 선을 지운다
- Signal Routing** → **Goto** 를 놓고 **Calc** 출력에 잇는다
- Goto** 를 더블클릭 → **Goto Tag** 를 **RESULT** 로
- Signal Routing** → **From** 을 놓고 **Display** 에 잇는다
- From** 을 더블클릭 → **Goto Tag** 를 **RESULT** 로
- Run** → **Display** 에 **12** 가 뜬다. $(10 - 4) \times 2 = 12$

C-4. From 은 여러 개 놓을 수 있다

- From** 을 하나 더 놓고 (태그도 **RESULT**) **Scope** 에 잇는다
- 선을 하나도 더 긋지 않고 같은 신호를 두 곳에서 읽었다

★ Goto/From 을 쓰는 이유

- 되먹임 선은 화면을 가로질러 되돌아온다. 그림이 금방 지저분해진다
- 태그로 이르면 선이 사라진다
- 주의:** 선이 안 보이니 흐름도 안 보인다. 되먹임처럼 꼭 필요한 곳에만 쓴다

🔪 태그 범위(Tag Visibility)

기본값 **local** 은 같은 화면 안에서만 통한다.

서브시스템 경계를 넘어가려면 **scoped** 나 **global** 이 필요하다. 이번 주차에는 **local** 로 충분하다.

확인

- ☐ **Calc** 서브시스템을 만들었고 포트 이름이 **a**, **b**, **y** 다
- ☐ **Display** 에 **12** 가 뜬다
- ☐ **From** 두 개가 같은 신호를 읽고 있다

D. 버스 (25분)

[D] 버스

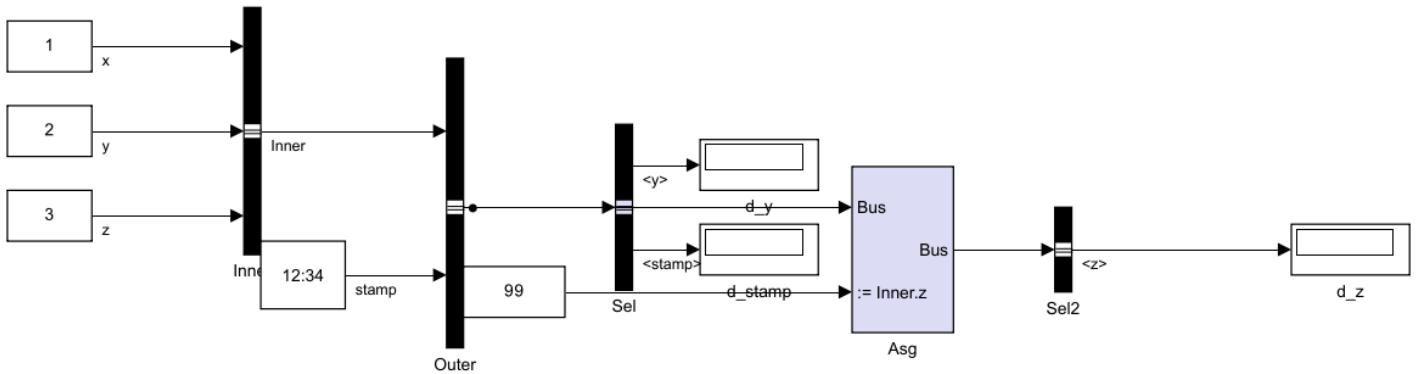
Bus Creator 는 여러 신호를 한 줄로 묶는다. 원소 이름은 신호선 이름을 따라간다.

Inner(x,y,z) 를 다시 Outer 안에 넣었다. 그래서 경로가 Inner.y 처럼 두 단이다.

Bus Selector 에서 원소를 고를 때 이 전체 경로를 그대로 적어야 한다.

Bus Assignment 는 버스를 통째로 받아 Inner.z 한 칸만 바꿔 내보낸다.

확인: d_y = 2, d_z = 99

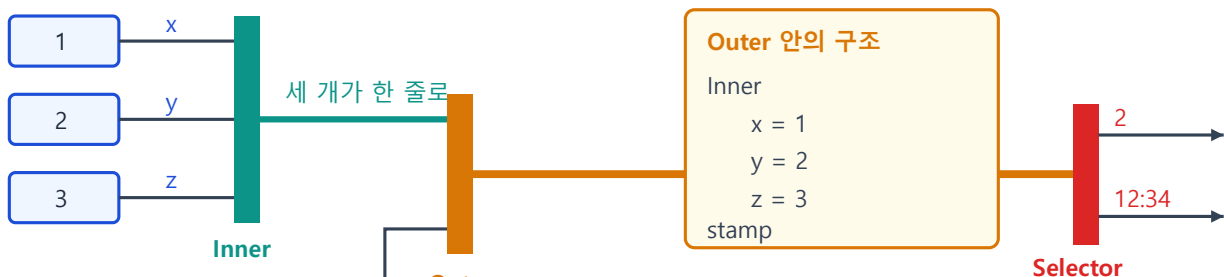


```
open_system('SB4_bus_todo')
```

D-1. 버스가 무엇인가

버스 — 여러 신호를 한 줄로 묶은 것

묶을 때 이름이 붙고, 풀 때 그 이름으로 꺼낸다



선 이름 = 원소 이름

꺼낼 때는 전체 경로를 적는다

y 라고만 적으면 못 찾는다. Inner.y 라고 적어야 한다
버스 안에 버스가 있으면 점(.)으로 계속 이어 붙인다

Bus Assignment 는 버스를 통째로 받아 한 칸만 바꿔 내보낸다 — 나머지는 그대로 지나간다

- 그림 읽는 법

요소	의미
굵은 검정 세로 막대	Bus Creator — 여러 신호를 한 줄로 묶음
굵은 선	버스 (여러 값이 함께 흐름)
Inner, stamp	버스 원소의 이름 — 신호선 이름을 따라감
Inner.y	버스 안의 버스에서 꺼낼 때 쓰는 전체 경로

- MATLAB 의 구조체와 똑같다고 보면 된다

```
Outer.Inner.x = 1
Outer.Inner.y = 2
Outer.Inner.z = 3
Outer.stamp    = 12.34
```

D-2. 이미 만들어져 있는 것

- 빈칸본에는 **Inner** 와 **Outer** 두 개의 Bus Creator 가 이미 연결되어 있다
- **Sel** 이 **stamp** 하나만 뽑아 **Display** 로 보내고 있다
- **Run** 해서 시각이 흐르는 것을 먼저 확인한다

D-3. 원소 하나 더 꺼내기

1. **Sel** (Bus Selector) 을 더블클릭
2. 왼쪽 목록에서 **Inner** 앞의 ► 를 눌러 펼친다 → **x**, **y**, **z** 가 보인다
3. **Inner.y** 를 고르고 가운데 **Select >>** 버튼을 누른다
4. **OK** → 블록에 출력 포트가 하나 늘어난다
5. **Display** 를 하나 더 놓고 잇는다
6. **Run** → **2** 가 뜨면 성공

⚠ 매 학기 반복적으로 문제가 발생하는 지점

중첩된 버스에서는 **y** 라고만 적으면 못 찾는다.

반드시 **Inner.y** 처럼 전체 경로를 써야 한다.

오류 메시지가 "신호를 찾을 수 없다"고 나오면 십중팔구 경로가 짧은 것이다.

D-4. 한 칸만 바꿔치기 — Bus Assignment

1. **Signal Routing** → **Bus Assignment** 를 캔버스에 놓는다
2. **Outer** 의 출력(굵은 선)에서 선을 하나 더 빼서 **Asg** 의 **1번 포트**에 잇는다
3. **Asg** 를 더블클릭 → 왼쪽에서 **Inner** 를 펼쳐 **Inner.z** 선택 → **Select >>** → **OK**
4. 블록에 **:= Inner.z** 라고 표시된 새 입력 포트가 생긴다
5. 이미 놓여 있는 **c_new** (값 99) 를 그 포트에 잇는다
6. **Bus Selector** 를 하나 더 놓아 **Asg** 출력에서 **Inner.z** 를 뽑고 **Display** 에 잇는다
7. **Run** → **99** 가 뜨면 성공

i Bus Assignment 가 하는 일

버스를 통째로 받아서 **지정한 칸만** 바꿔 내보낸다.

나머지 원소는 손대지 않고 그대로 지나간다.

6주차에 ROS 메시지를 채울 때 이 블록을 쓰게 된다.

확인

- ☐ **Inner.y** 를 뽑아 **2** 가 뜬다
- ☐ **Bus Assignment** 로 바꾼 **Inner.z** 가 **99** 로 뜬다

E. PID · 포화 · 안티와인드업 (25분)

[E] 이산 PID · 포화 · 안티와인드업

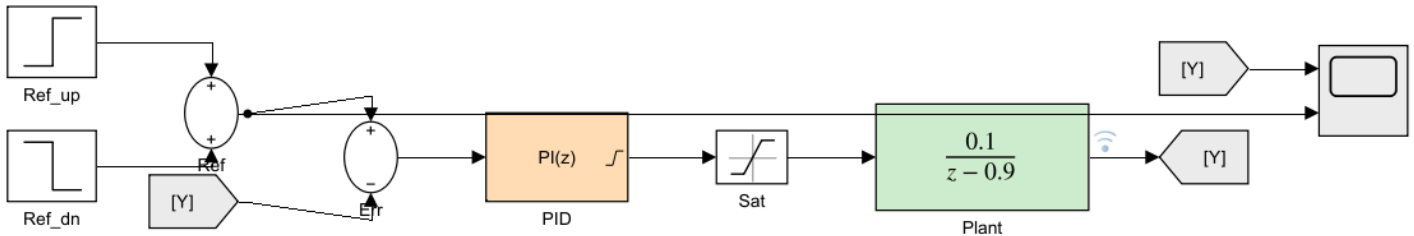
Plant 는 아무 의미 없는 장난감 전달함수다. 물리와 상관없다.

정상 이득이 1 이고 포화가 ± 1 이라 출력은 1 을 못 넘는다.

그런데 1~10초 목표는 2 다. 도달할 수 없는 목표라 적분기가 계속 부른다.

10초에 목표를 0.5 로 내렸을 때 얼마나 빨리 따라오는지가 관전 포인트.

PID 대화상자에서 Anti-windup method 를 none $\leftrightarrow$ clamping 으로 바꿔 볼 것.



i 이번 주차에는 제어 이론을 배우지 않는다

배우는 것은 블록을 어떻게 놓고 대화상자를 어떻게 채우는가뿐이다.

Plant 는 아무 의미 없는 장난감 전달함수다. 배와 상관없다.

게인을 왜 그 값으로 정하는지는 8주차에 다룬다.

```
open_system('SB5_pid_todo')
```

E-0. 이 모델의 구조

블록	하는 일
Ref_up , Ref_dn , Ref	목표값을 만든다 — 0 → 2 (1초) → 0.5 (10초)
Err	목표 - 현재 = 오차
PID	오차를 보고 얼마나 밀지 정함
Plant	밀면 반응하는 무언가. 정상 이득이 1
Goto/From [Y]	출력을 다시 앞으로 되돌림

★ 목표 2 는 도달할 수 없는 값이다

Plant 의 정상 이득이 1 이고 나중에 불일 포화가 ± 1 이라,

출력은 아무리 해도 1 을 넘지 못한다.

이 "도달 못 하는 상태"가 오래 이어지는 것이 뒤에 나올 문제의 씨앗이다.

E-1. 기본 설정으로 실행 — P 제어만

- Run 하고 Scope 를 연다

구간	목표	실제 출력
1~10초	2	1.0 에서 멈춤
10초~	0.5	0.25 에서 멈춤

- 어느 쪽도 목표를 못 맞춘다. 이것이 정상상태 오차다
- P 제어만으로는 오차가 0 이 되면 미는 힘도 0 이 되어 버린다

E-2. I 를 더한다

1. PID 를 더블클릭
 2. Controller 를 PI 로 바꾼다
 3. Integral (I) 에 2 를 넣는다
 4. Sample time 이 0.05 인지 확인
 5. OK → Run
- 이번엔 목표를 맞춘다 (2 는 여전히 못 가지만 0.5 는 정확히 맞춤)

E-3. 포화를 끼워 넣는다

- 실제 장비에는 한계가 있다. 그 한계를 흉내 내는 블록이 Saturation 이다
1. PID 와 Plant 사이의 선을 클릭하고 Delete
 2. Discontinuities → Saturation 을 그 자리에 놓는다
 3. 더블클릭 → Upper limit 1, Lower limit -1 → OK
 4. PID → Sat → Plant 로 다시 잇는다
 5. Run

⚠ 예상과 다른 결과가 나타나는 지점

10초에 목표가 0.5 로 내려갔는데도 출력이 한참 동안 1 에 머문다.
Scope 를 끝까지(30초) 보면 거의 안 내려온다.
이것이 적분 와인드업이다.

- 왜 그런가

시각	일어나는 일
1~10초	목표 2 에 못 닿음 → 오차가 계속 남음
그동안	적분기가 오차를 계속 쌓는다. 아주 큰 값이 된다
10초	목표가 0.5 로 내려감
10초 이후	쌓인 값이 다 빠질 때까지 계속 밀어 댄다

E-4. 안티와인드업을 켜다

1. PID 를 더블클릭
 2. Output saturation 탭에서 Limit output 을 체크
 3. Upper limit 1, Lower limit -1
 4. Anti-windup method 를 clamping 으로
 5. OK → Run
- 이제 10초 직후 바로 0.5 로 내려온다

E-5. 두 결과를 나란히 비교하기

- Anti-windup method 를 none ↔ clamping 으로 바꿔 가며 두 번 돌린다
- 참고 수치 (이 모델에서 실제로 측정한 값)

안티와인드업	목표 0.5 에 닿은 시각	10초 이후 지연
none	29.8초	19.8초
clamping	12.6초	2.6초

- 차이가 약 17초다. 눈으로 바로 보인다

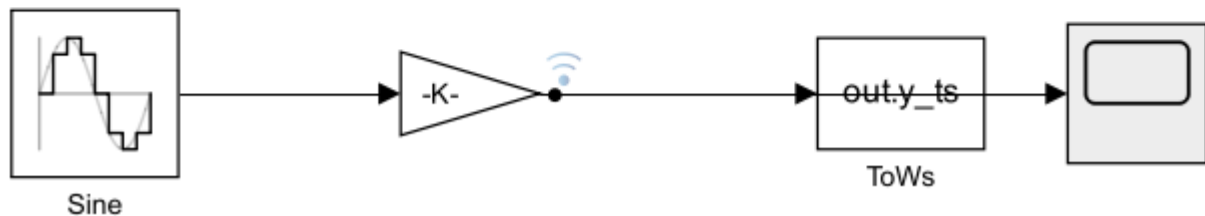
확인

- ☐ P 만일 때 정상상태 오차를 봤다
- ☐ 포화를 넣었더니 10초 이후 안 내려오는 것을 봤다
- ☐ `clamping` 을 켜니 바로 내려오는 것을 봤다

F. 파라미터 · 로깅 · Data Inspector (25분)

[F] 파라미터 · 로깅

Sine 의 진폭이 `A_sig`, Gain 이 `K_gain`, 샘플 타임이 `Ts` 로 되어 있다.
숫자가 아니라 변수 이름이다. 값은 MATLAB 작업공간에서 읽어 온다.
먼저 `>> SB_setup` 을 실행해야 돌아간다.
Gain 출력에 `y` 라는 이름으로 로깅이 켜져 있다 -> `out.logout`
To Workspace 는 같은 신호를 `y_ts` 로 따로 내보낸다.
실행: `>> SB_setup; out = sim('SB6_param_done'); SB_plot(out)`



```
open_system('SB6_param_todo')
```

F-1. 블록에 숫자 대신 변수 이름 쓰기

- 지금은 값이 숫자로 박혀 있다. 바꾸려면 블록을 하나하나 열어야 한다
 - 대신 **변수 이름**을 적으면, 그 값은 MATLAB 작업공간에서 읽어 온다
1. `Sine` 더블클릭 → **Amplitude** 를 `A_sig` 로, **Sample time** 을 `Ts` 로
 2. `Gain` 더블클릭 → 값을 `K_gain` 으로
 3. **Run** 을 눌러 본다

⚠ 오류를 의도적으로 발생시키는 단계

아래와 같은 메시지가 뜬다. 당황하지 말 것.

```
'SB6_param_todo/Gain'에서 파라미터 'Gain'에 대한 설정이 유효하지 않습니다.  
'SB6_param_todo/Sine'에서 파라미터 'SampleTime'에 대한 설정이 유효하지 않습니다.
```

변수가 아직 없어서 나는 오류다.

4. MATLAB 명령창에서

```
SB_setup
```

- 정상 출력

```
SB_setup 완료: A_sig = 1, K_gain = 2, Ts = 0.05
```

5. 다시 **Run** → 이제 돌아간다

★ 이 방식의 장단점

- **좋은 점:** 값을 한 파일(`SB_setup.m`)에서 모두 관리. 실험 조건 바꾸기 쉬움
- **나쁜 점:** 모델만 보서는 **실제 값이 안 보인다**
- 그래서 `SB_setup.m` 에 주석을 잘 달아 두어야 한다

F-2. 신호에 이름 붙이고 로깅하기

1. `Gain` 의 출력 신호선을 클릭해서 선택
2. 우클릭 → **Properties** → **Signal name** 에 `y` 입력 → OK
3. 다시 그 선을 우클릭 → **Log Selected Signals** 클릭
4. 선 위에 작은 파란 안테나 표시가 생긴다 — 로깅이 켜졌다는 뜻

F-3. To Workspace 블록

- 같은 신호를 다른 방법으로도 꺼낼 수 있다
- 1. **Sinks** → **To Workspace** 를 놓는다
- 2. 더블클릭 → **Variable name** 을 `y_ts` 로 → OK
- 3. `Gain` 출력에서 `Ctrl` 드래그로 선을 빼서 잇는다

F-4. 스크립트로 실행하고 결과 꺼내기

```
SB_setup
out = sim('SB6_param_todo');
```

- 로깅한 신호 꺼내기

```
sig = out.logsout.getElement('y');
plot(sig.Values.Time, sig.Values.Data)
```

- 또는 준비된 스크립트로

```
SB_plot(out)
```

- 정상 출력

```
[신호 로깅] out.logsout.getElement('y')
표본 개수 : 201
최댓값    : 2.0000
최솟값    : -2.0000
```

! 표본이 201개인 이유

0.05초 간격으로 10초를 돌면 $10 / 0.05 + 1 = 201$ 이다.
숫자가 안 맞으면 솔버 설정이 틀린 것이다.

F-5. Data Inspector 로 겹쳐 보기

- 조건을 바꿔 두 번 돌린 결과를 한 그래프에 겹쳐 보는 도구다
- 1. MATLAB 명령창에서 `K_gain = 2` 로 두고 **Run**
- 2. `K_gain = 5` 로 바꾸고 다시 **Run**

```
K_gain = 5;
```

3. 툴스트립의 **Data Inspector** 버튼을 누른다
4. 왼쪽에 **Run 1**, **Run 2** 두 개가 보인다
5. 두 Run 의 **y** 를 **모두 체크**하면 한 그래프에 겹쳐 그려진다
6. 그래프 위를 클릭하면 **커서**가 생겨 그 시각의 값을 정확히 읽을 수 있다

🔧 이 도구는 8주차에서도 사용한다

8주차부터 "상승시간 몇 초, 오버슈트 몇 %" 같은 표를 만들게 된다.
그 숫자를 눈대중으로 읽지 말고 **여기 커서로 읽어서 적으면 된다.**

확인

- ☐ 변수가 없을 때 나는 오류 메시지를 직접 봤다
 - ☐ **SB_setup** 후 정상 실행된다
 - ☐ **out.logout** 에서 신호를 꺼내 그래프를 그렸다
 - ☐ Data Inspector 에서 두 Run 을 겹쳐 봤다
-
-

2일차 · 7~9주차에서 실제로 쓰는 블록들

★ 이후 절은 2일차(3시간) 다

1일차(AF)가 Simulink 문법이라면, 2일차(GL)는 본 과목의 모델을 읽기 위한 최소 어휘다.
아래 표의 블록들은 전부 7~9주차 모델 안에 실제로 들어 있다.

이 블록들이 어디에 쓰이는가

절	블록	본 과목에서 쓰이는 곳
G	Mux Demux Selector	운동모델의 상태 6개를 한 선으로 묶고 다시 툰다
H	Unit Delay	웨이포인트 번호 되먹임(7주), 대수 루프 차단(9주)
H	샘플타임 · Rate Transition	제어 0.05 s ↔ 센서 다른 주기
I	Integrator	운동방정식 적분 — 배의 위치가 여기서 나온다
I	Transfer Fcn	모터 1차 지연 $1/(\tau s + 1)$ (7주 Thrusters)
I	적분기 출력 제한	물리 한계가 있는 상태
J	Switch Multiport Switch	유도법칙 같아 끼우기(9주 ModeSwitch)
J	Stop Simulation	임무 종료(8·9주)
K	Enabled Subsystem	"새 메시지가 왔을 때만 계산" (ROS IsNew)
L	Mask	PID 블록처럼 다이얼로그를 가진 서브시스템

2일차 모델 만들기

```
cd('<배포 폴더>/W06_0_simulink')
build_w06_1_models
```

- SB7 ~ SB12 의 _done / _todo 12개가 만들어진다
- 1일차 모델(SB1 ~`SB6 )은 build\_w06\_0\_models` 가 만든다

G. 신호 묶기와 풀기 — Mux · Demux · Selector (25분)

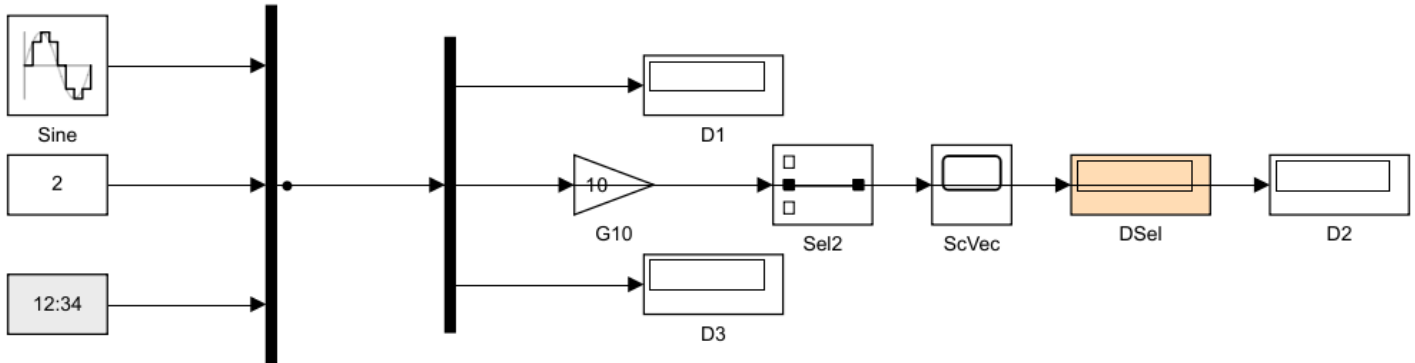
[G] 신호 묶기와 풀기

Mux : 여러 신호를 한 선(벡터)으로 묶는다. 선이 굵게 표시된다

Demux: 벡터를 다시 날개로 푼다

Selector: 벡터에서 원하는 번호만 뽑는다 (여기서는 2번 = 상수 2)

Gain 은 벡터 전체에 한꺼번에 걸린다 -> 세 값이 모두 10배



G-1. 왜 필요한가

- 배의 상태는 여섯 개다 — `u v r N E ψ`
- 선을 여섯 가닥 끝면 화면이 빠르게 엉킨다
- 한 선(벡터)으로 묶어서 옮기고, 쓸 곳에서 푼다

블록	하는 일	기억법
Mux	여러 신호 → 벡터 하나	묶는다
Demux	벡터 → 날개 신호	푼다
Selector	벡터에서 원하는 번호만 뽑는다	골라낸다
Reshape	벡터의 모양을 바꾼다 (<code>[6x1]</code> ↔ <code>[1x6]</code>)	모양만 바꾼다

굵은 선은 벡터 신호를 뜻한다

Mux 를 지나면 신호선이 굵게 그려진다. 시각적으로 즉시 구분된다.

굵기가 안 보이면 `Debug → Information Overlays → Signal Dimensions` 를 켜다.

G-2. 벡터에는 연산이 통째로 걸린다

- `Gain(10)` 하나가 세 신호 모두에 걸린다
- `Sum`, `Saturation` 도 마찬가지
- 이것이 벡터를 쓰는 진짜 이유다 — 블록 수가 줄어든다

G-3. 해 볼 것 (`SB7_signal_todo`)

1. Mux (입력 3) 에 `Sine`, `상수 2`, `Digital Clock` 을 연결
2. `Gain(10)` → `Scope` — 세 곡선이 한꺼번에 10배가 되는지 확인
3. Demux (출력 3) 로 풀어 `Display` 세 개에 연결
4. Selector 의 `Indices` 를 2 로 → `상수 2` 만 나오는지 확인
5. Mux 출력선을 우클릭 → `Signal Properties` → 이름을 `state` 로

⚠ 벡터와 버스는 다르다

구분	벡터	버스
원소	같은 타입·같은 단위	서로 달라도 됨
접근	번호 (Selector)	이름 (Bus Selector)
쓰는 곳	상태 [u v r N E psi]	ROS 메시지 (pose.position.x)

1일차 D절의 버스와 헛갈리지 말 것. ROS 메시지는 **항상 버스**다.

H. 이산 시스템 — 샘플타임과 Unit Delay (35분)

[H] 이산 시스템

위: Unit Delay 로 만든 누적기. $y[k] = y[k-1] + Ts \cdot u[k]$

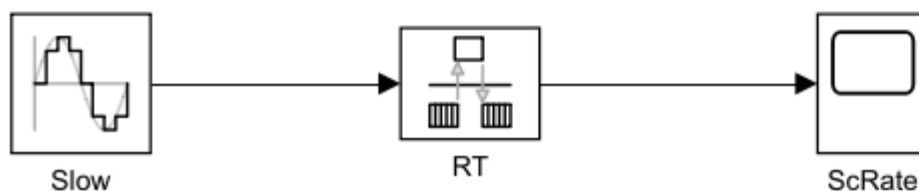
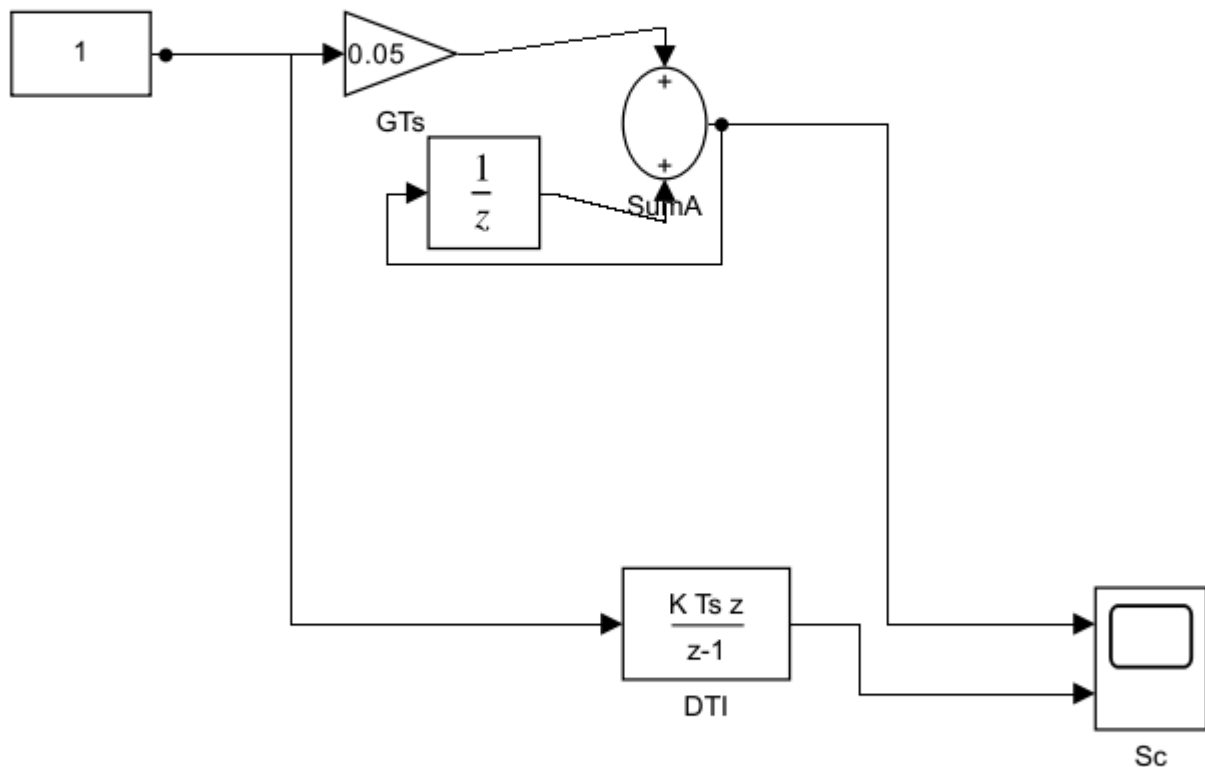
-> 10초 뒤 값이 10 이 되면 맞다 (1 을 0.05 씩 200번 더함)

아래: Discrete-Time Integrator (Backward Euler) 는 같은 일을 하는 기성 블록

두 곡선이 정확히 겹친다. 적분법을 Forward Euler 로 바꾸면 한 스텝(0.05) 어긋난다

맨 아래: 0.5 s 신호를 0.05 s 쪽으로 넘길 때 Rate Transition 이 필요하다

샘플타임 색 표시가 켜져 있다 (Debug -> Information Overlays -> Sample Time)



H-1. 샘플타임이란

- 이 블록이 몇 초마다 한 번 계산되는가
- 본 과목은 제어 주기를 `Ts_ctrl = 0.05 s` (20 Hz) 로 통일한다
- 블록마다 다른 주기를 줄 수 있고, 그러면 멀티레이트 모델이 된다

🔍 샘플타임의 색상 표시

Debug → Information Overlays → Sample Time → Colors

같은 주기끼리 같은 색이 된다. 주기가 섞인 곳을 시각적으로 확인하는 방법이다.

`SB8_discrete_done` 은 이 표시가 켜진 채로 저장돼 있다.

H-2. Unit Delay — 한 스텝 전 값을 기억한다

$$y[k] = u[k-1]$$

- 이 블록 하나로 상태를 가진 계산을 만들 수 있다
- 누적기(적분기)를 손으로 만들면 이렇게 된다

$$y[k] = y[k-1] + Ts * u[k]$$

- 블록으로는 `Sum(++)` → `Unit Delay` → 다시 `Sum` 의 두 번째 입력
- 실측 (`SB8_discrete_done` , 10초, `Ts = 0.05` , 입력 1)

방식	10초 뒤 값
Unit Delay 로 만든 누적기	10.0500
<code>Discrete-Time Integrator</code> (Backward Euler)	10.0500
두 곡선의 최대 차이	0

! 왜 10.00 이 아니라 10.05 인가

0초부터 10초까지 0.05 s 간격 샘플은 201개다. $201 \times 0.05 = 10.05$.

적분법을 `Forward Euler` 로 바꾸면 정확히 한 스텝(0.05) 어긋난다.

한 스텝 차이가 어디서 오는지 설명할 수 있어야 한다.

H-3. Unit Delay 를 이용한 대수 루프 차단 (중요)

- 되먹임 고리 안에 지연이 하나도 없으면 Simulink 는 계산 순서를 정하지 못한다

Algebraic loop containing block ... detected

- 원인: `y` 를 구하려면 `y` 가 필요한 구조
- 해법: 고리 안에 `Unit Delay` 를 하나 넣는다 → "한 스텝 전 값"으로 끊는다
- 9주차에서 실제로 겪는다. 상태(`mode`)가 자기 자신에게 되먹임되기 때문

🔍 대수 루프의 시각적 확인

Debug → Diagnostics → Algebraic Loops → Highlight 를 쓰면

고리에 해당하는 블록이 빨간색으로 표시된다.

H-4. 멀티레이트와 Rate Transition

- 0.5 s 신호를 0.05 s 쪽으로 그대로 연결하면 경고가 난다
- `Rate Transition` 블록이 주기를 안전하게 넘겨 준다 (값을 붙잡아 둔다)

- ROS 센서는 주기가 제각각이라 실전에서 자주 만난다

H-5. 해 볼 것 (SB8_discrete_todo)

1. Gain(0.05) → Sum(++) → Unit Delay 되먹임으로 누적기 만들기
2. 10초 뒤 값이 10.05 인지 확인
3. Discrete-Time Integrator 를 놓고 겹쳐 보기 (적분법을 Backward Euler 로)
4. Unit Delay 를 빼 보고 어떤 오류가 나는지 확인 — 오류 메시지를 그대로 적어 둘 것
5. Rate Transition 없이 0.5 s 신호를 이어 보고 경고 확인
6. 샘플타임 색 켜기

I. 연속 시스템 — Integrator · Transfer Fcn · 솔버 (30분)

[I] 연속 시스템

위: Transfer Fcn $1/(0.3s+1)$ — 1차 지연. 7주차 모터 모델이 바로 이것이다

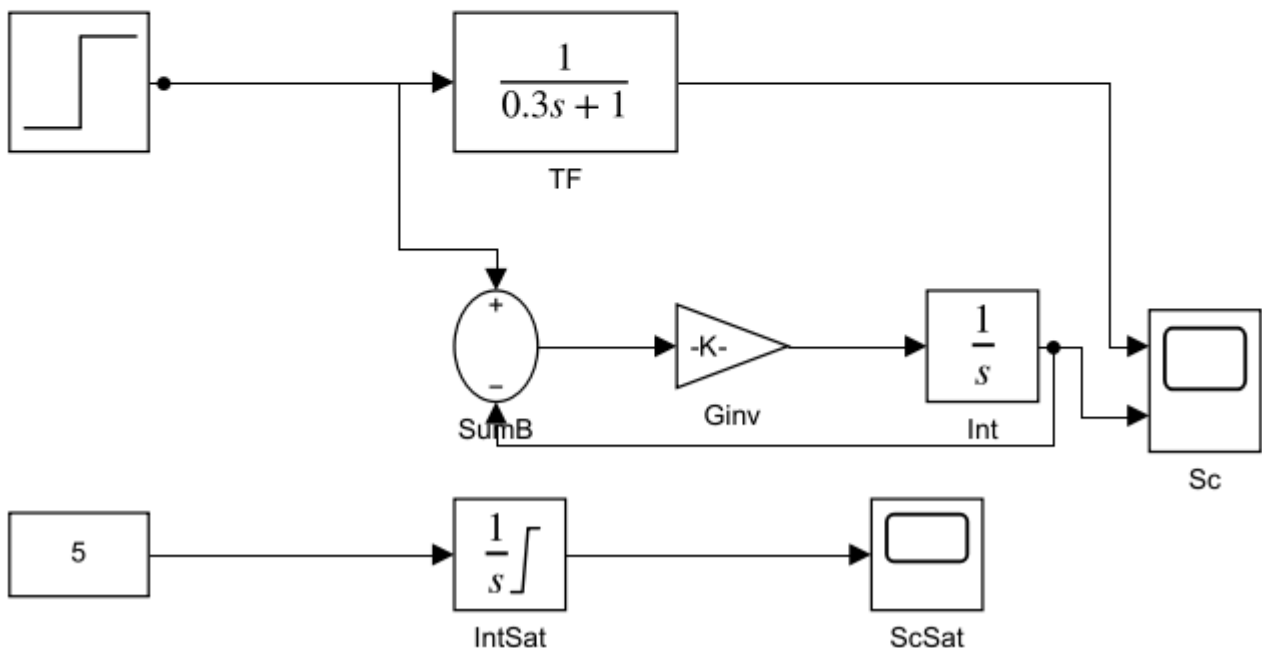
가운데: 같은 식을 Integrator 로 직접 짤 것. $dy/dt = (u - y)/\tau$

두 곡선이 겹쳐야 한다. 겹치지 않으면 배선이나 계인을 확인

아래: Integrator 에 출력 한계 +3 을 걸었다 (Limit output)

물리적 한계가 있는 상태는 이렇게 막는다. 안 막으면 무한히 커진다

솔버가 ode45(가변 스텝)로 바뀐 것에 주의 — 연속 상태가 있으면 필요하다



I-1. 배의 위치는 적분에서 나온다

- 운동방정식은 미분방정식이다

$$m \cdot du/dt = X - (X_u + X_{uu} \cdot |u|) \cdot u$$

$$dN/dt = u \cdot \cos(\psi) - v \cdot \sin(\psi)$$

- Simulink 에서 미분방정식을 푸는 방법은 한 가지 — du/dt 를 만들어 Integrator 에 넣는다
- 7주차 MotionModel 안이 정확히 이 모양이다

I-2. 1차 지연 — 모터가 즉시 돌지 않는 이유

$$dy/dt = (u - y) / \tau \quad \Leftrightarrow \quad Y(s)/U(s) = 1/(\tau s + 1)$$

- 같은 식을 두 가지로 만들 수 있다

방법	블록
전달함수로	<code>Transfer Fcn</code> , 분자 <code>[1]</code> , 분모 <code>[\tau 1]</code>
직접	<code>Sum(+)</code> → <code>Gain(1/\tau)</code> → <code>Integrator</code> → 출력을 <code>Sum</code> 으로 되먹임

- 실측 (`SB9_continuous_done`, $\tau = 0.3$, 계단 입력 $t = 1$ s)

항목	값
두 방식의 최대 차이	6.7×10^{-16} (수치오차)
63.2 % 도달 시간	$0.300 \text{ s} = \tau$
최종값	1.0000

★ τ 의 의미

63.2 % 에 도달하는 시간이다. 5 τ 면 거의 다 왔다고 본다.
7주차 모터 시상수 `tau_n = 0.30 s` 도 같은 뜻이다.

I-3. 물리적 한계가 있는 상태

- 실제 상태에는 한계가 있다 — 추력, 조향각, 탱크 수위
- `Integrator` 의 **Limit output** 을 켜고 상·하한을 준다
- 실측: 상수 5 를 계속 적분해도 출력이 3.0000 에서 멈춘다

⚠ 한계를 설정하지 않으면 오류 없이 발산한다

오류가 나지 않는다. 값이 계속 커진다.
그리고 그 상태로 제어기에 들어가면 와인드업이 된다 (E절).

I-4. 솔버 — 연속 상태가 있으면 바꿔야 한다

솔버	언제
<code>FixedStepDiscrete</code>	이산 블록만 있을 때. 1일차 모델 전부
<code>ode3</code> (고정 스텝)	연속 상태가 있고 실시간·코드생성이 필요할 때
<code>ode45</code> (가변 스텝)	연속 상태가 있고 정확도가 중요할 때. 기본값

- `SB9` 는 `ode45` 로 저장돼 있다. 고정 스텝 0.05 로 바꿔 보면 곡선이 살짝 달라진다
- 가변 스텝은 실시간 연동에 쓰지 않는다 — 6주차 이후 VRX 모델이 전부 고정 스텝인 이유

! 영점 교차 검출 (Zero-Crossing)

`Saturation`, `Switch`, `abs` 같은 블록은 값이 꺾이는 순간이 있다.
가변 스텝 솔버는 그 순간을 정확히 찾아 스텝을 끊는다.
이것 때문에 시뮬레이션이 느려지면 블록별로 끌 수 있다.

I-5. 해 볼 것 (`SB9_continuous_todo`)

- `Transfer Fcn` 분모 `[0.3 1]` 로 두고 계단 응답 보기

2. 63 % 도달 시각이 **0.3 s** 인지 커서로 재기
 3. 같은 것을 **Integrator** 로 직접 만들어 겹쳐 보기
 4. **Integrator** 에 **Limit output** 을 걸어 ± 3 으로 막기
 5. 솔버를 **ode45** ↔ 고정 스텝 **0.05** 로 바꿔 가며 차이 관찰
 6. tau 를 0.05 로 줄이고 고정 스텝 0.05 로 돌려 보기 — 왜 이상해지는가?
-

[J] 판단 블록

Relational Operator: 두 값을 비교해 참(1)/거짓(0)을 낸다

Switch: 가운데 입력이 조건을 만족하면 위, 아니면 아래를 내보낸다

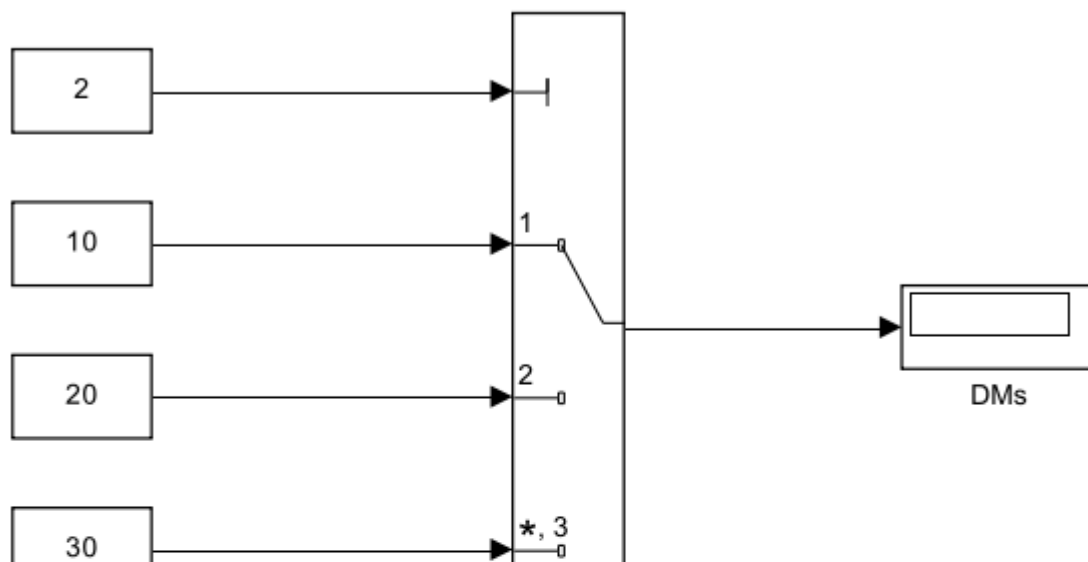
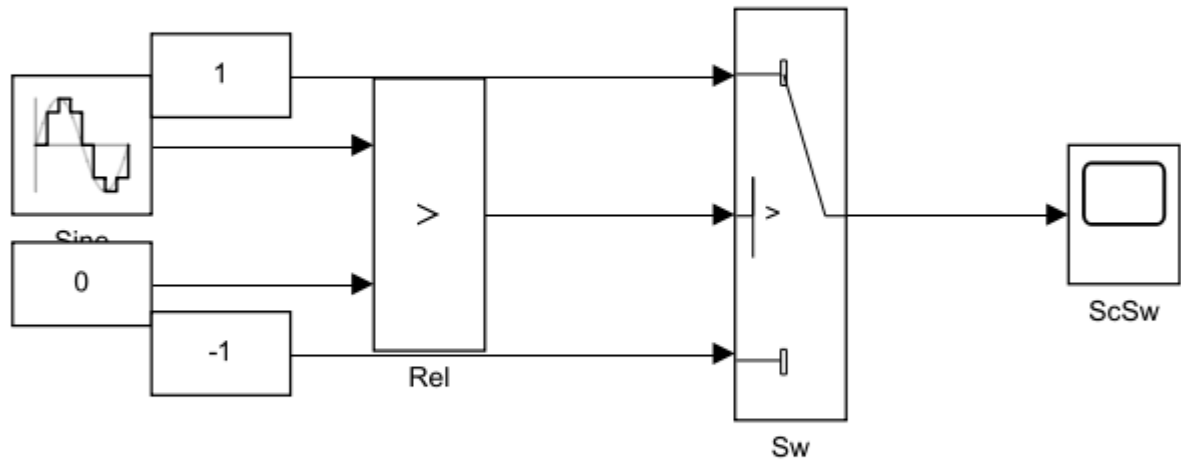
-> 사인파의 부호에 따라 +1 / -1 이 나온다

Multiport Switch: 첫 입력이 번호. **Mode=2** 이므로 두 번째 값 20 이 나온다

-> 9주차 ModeSwitch(유도법칙 알아 끼우기)가 바로 이 블록이다

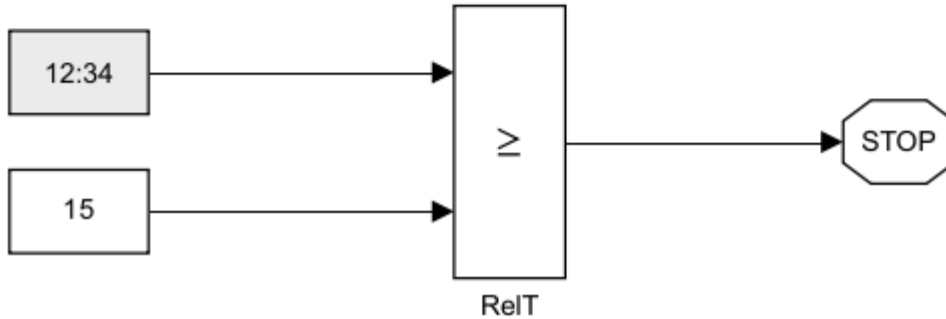
Stop Simulation: 입력이 0 이 아니면 시뮬레이션을 멈춘다

정지 시간을 60 으로 뒀지만 15초에서 멈춘다. 8·9주차 임무 종료가 이 방식



[]

MSw



J-1. if 문을 블록으로

블록	MATLAB 으로 치면
Relational Operator	<code>a > b</code>
Logical Operator	<code>AND , OR , NOT</code>
Switch	<code>if cond, y = a; else, y = b; end</code>
Multiport Switch	<code>switch k, case 1 ... case 2 ...</code>

🔍 MATLAB Function 블록과의 사용 구분

- 조건이 **한두 개**면 블록이 낫다 — 신호 흐름이 시각적으로 확인된다
- 조건이 **여러 개** **얹히면** MATLAB Function 이 낫다 — 7주차 **Guidance** 가 그렇다
- **상태가 있으면** Stateflow (9주차)

J-2. Switch 의 판단 기준

- 가운데(2번) 입력이 조건 신호다
- **Criteria** 를 `u2 > Threshold` 로 두고 **Threshold** 를 0.5 로 하면
 - 조건이 참(1) → **위쪽(1번)** 입력이 나간다
 - 거짓(0) → **아래쪽(3번)** 입력이 나간다

J-3. Multiport Switch — 9주차 모드 전환의 원리

- 맨 위 입력이 **번호**, 나머지가 후보들
- **Mode = 2** 면 두 번째 값이 나간다 (실측 20)
- 9주차 **ModeSwitch** 가 이 블록이다
 - `mode = 1` → LOS 유도 의 `psi_ref`
 - `mode = 2` → 벡터필드 유도 의 `psi_ref`

★ 둘 다 계산하고 출력만 고른다

안 쓰는 쪽을 꺼 두는 게 아니다. 두 유도법칙이 **항상 함께 돈다**.
그래야 모드가 바뀌는 순간 값이 튀지 않는다 (bumpless transfer).

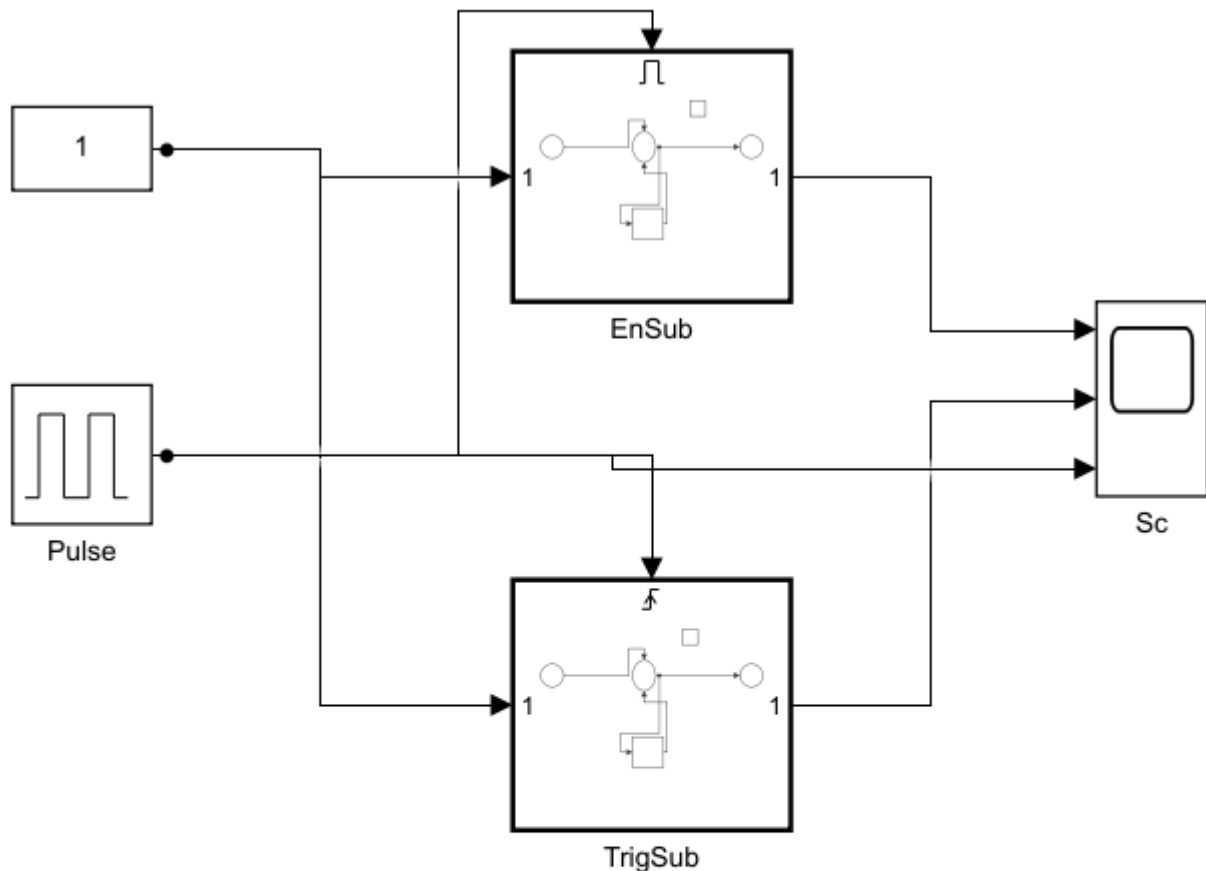
J-4. Stop Simulation — 임무가 끝나면 멈춘다

- 입력이 **0** 이 **아니면** 시뮬레이션이 그 자리에서 끝난다
- **SB10** 은 정지 시간이 60 인데 **15초에 멈춘다** (실측 종료 시각 15.00 s)
- 8주차: 정해진 바퀴 수를 다 돌면 / 9주차: 마지막 웨이포인트에 도착하면

J-5. 해 볼 것 (**SB10_logic_todo**)

1. **Relational Operator(>)** 로 Sine 과 0 을 비교 → **Switch** → 사각파 만들기
2. **Multipoint Switch** 에 상수 10/20/30 을 붙이고 **Mode** 를 1:2:3 으로 바꿔 확인
3. **Digital Clock >= 15** → **Stop Simulation** 을 걸고 **15초에 멈추는지** 확인
4. **Mode** 에 **0** 이나 **4** 를 넣으면 어떻게 되는가? 오류 메시지를 적어 둘 것

K. 조건부 실행 — Enabled · Triggered Subsystem (25분)



[K] 조건부 실행 서브시스템

두 서브시스템 안에는 똑같은 누적기가 들어 있다 (Unit Delay + Sum)

Enabled : enable 이 1 인 동안 **매 스텝** 돈다. 0 이면 값을 그대로 유지

Triggered: 신호가 **올라가는 순간에만 한 번** 돈다

-> Enabled 는 계단처럼 쭉쭉 오르고, Triggered 는 한 칸씩만 오른다

6주차의 ROS Subscribe 는 isNew 를 enable 로 쓰면 딱 이 구조가 된다

"새 메시지가 왔을 때만 계산한다"

K-1. 항상 둘 필요는 없다

종류	언제 도는가
보통 서브시스템	매 스텝
Enabled	enable 신호가 1 인 동안 매 스텝
Triggered	신호가 올라가는 순간에만 한 번
Enabled and Triggered	둘 다 만족할 때

K-2. 실측으로 보는 차이

- 두 서브시스템 안에 똑같은 누적기를 넣고, 같은 펄스(주기 4 s, 듀티 50 %)를 준다
- 20초 돌린 결과

서브시스템	최종 누적값	왜
Enabled	201	켜져 있는 10초 동안 매 스텝(0.05 s) 돌았다
Triggered	5	20초 동안 올라간 순간이 5번뿐이다

★ 이 차이가 ROS 와 직결된다

6주차 `Subscribe` 블록의 `IsNew` 출력을 `enable` 로 쓰면
"새 메시지가 왔을 때만 계산한다" 가 그대로 구현된다.
안 그러면 같은 메시지를 몇 번씩 다시 계산한다.

K-3. 꺼져 있을 때 상태는 어떻게 되나

- `Output` 를 열면 `Output when disabled` 가 있다
 - `held` — 마지막 값을 붙잡고 있다 (기본)
 - `reset` — 초기값으로 돌아간다
- 서브시스템 안의 상태도 `States when enabling` 으로 정할 수 있다
- 제어기를 껐다 켤 때 적분값을 유지할지 버릴지가 이 설정이다

K-4. 해 볼 것 (`SB11_enabled_todo`)

1. `Enabled Subsystem` 안에 누적기(`Sum` + `Unit Delay`) 만들기
2. 같은 누적기를 넣은 `Triggered Subsystem` 하나 더
3. 두 출력과 펄스를 한 Scope 에 겹쳐 보기 — 201 대 5 를 눈으로 확인
4. `Output when disabled` 를 `held` ↔ `reset` 으로 바꿔 차이 적기

L. 재사용 — Mask · 라이브러리 · 코드로 만들기 (30분)

[L] 마스크로 재사용하기

같은 서브시스템 두 개가 τ 만 다르게 들어 있다 (0.1 s / 1.0 s)

블록을 더블클릭하면 **다이얼로그 상자**가 뜬다. 안을 열지 않아도 값을 바꿀 수 있다

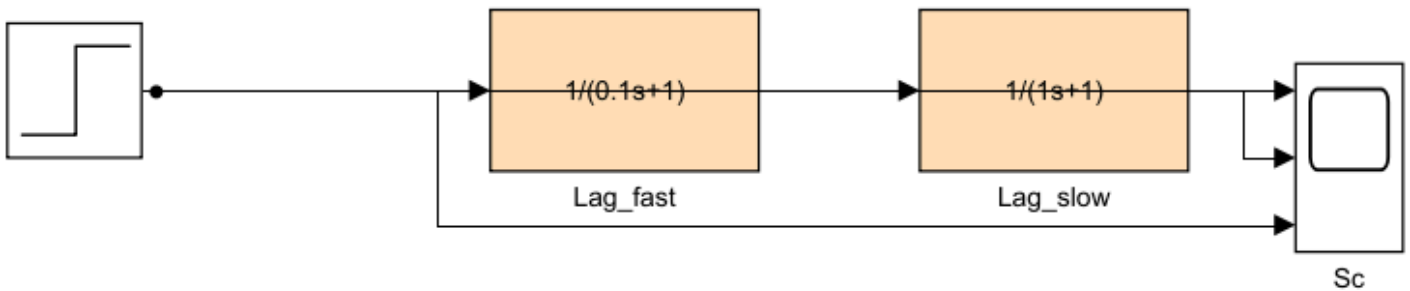
-> 마스크(Mask). Simulink 의 PID 블록도 이렇게 만들어져 있다

마스크를 보려면 우클릭 -> Mask -> Edit Mask

안을 보려면 우클릭 -> Mask -> Look Under Mask

이 수업의 모든 모델은 build_wXX_models.m 이 코드로 만든다.

마스크도 Simulink.Mask.create 로 코드에서 만들 수 있다



L-1. Mask — 서브시스템에 다이얼로그를 붙인다

- 같은 구조를 값만 바꿔 여러 번 쓰고 싶다
- 서브시스템 우클릭 → **Mask** → **Create Mask**
 - Parameters** 탭 — 이름 `tau`, 프롬프트 `시상수 [s]`, 기본값 `0.3`
 - 안쪽 블록에서 `1/tau` 처럼 그 이름을 그대로 쓴다
 - Icon** 탭 — `disp(sprintf('1/(%gs+1)', tau))` 로 아이콘에 식 표시
- 결과: 더블클릭하면 안을 열지 않고도 값을 바꿀 수 있다
- 실측 (`SB12_reuse_done` , 같은 마스크 블록 두 개)

블록	<code>tau</code>	63 % 도달 시간
<code>Lag_fast</code>	0.1	0.100 s
<code>Lag_slow</code>	1.0	1.000 s

! 익숙한 그 블록들이 전부 마스크다

`PID Controller` , `Saturation` , `Transfer Fcn` — 더블클릭하면 다이얼로그가 뜬다.

우클릭 → **Mask** → **Look Under Mask** 로 안을 볼 수 있다.

학습자가 만든 것도 똑같이 동작한다.

L-2. 라이브러리 — 여러 모델이 같은 블록을 공유한다

구분	복사·붙여넣기	라이브러리 링크
고칠 때	전부 찾아다녀야 한다	원본 하나만 고치면 된다
표시	일반 블록	왼쪽 아래에 화살표
끊기	—	우클릭 → Library Link → Disable

```
new_system('my_usv_lib','Library');
open_system('my_usv_lib');
```

- 블록을 끌어다 놓고 저장하면 끝이다
- 연구실 모델(`VRX_tilt4_controller_full.slx`)에도 링크 블록이 들어 있다

L-3. Model Reference — 모델을 블록처럼 쓴다

구분	Subsystem	Model Reference
저장	부모 모델 안에	별도 <code>.slx</code> 파일
따로 실행	안 됨	됨
팀 작업	충돌 남	파일이 나뉘어 안전
컴파일	매번	한 번 하고 재사용 (빠름)

- Term Project 에서 팀원이 각자 만든 부분을 합칠 때 이 방식을 쓴다

L-4. 코드로 모델을 만든다 — 본 과목의 방식

★ 본 과목의 모든 모델은 손으로 그리지 않았다

`build_w06_1_models.m` 이 지금 연 12개를 전부 만들었다.
7~9주차 모델도 `build_w07_models.m` 같은 스크립트가 만든다.

- 왜 그렇게 하는가

이유	설명
복구	학생이 마음껏 만져도 한 줄로 원상복구
일관성	22개 모델의 배치·색·배선 규칙이 똑같아진다
버전 관리	<code>.slx</code> 는 diff 가 안 된다. <code>.m</code> 은 된다

- 핵심 함수 네 개만 알면 된다

```
new_system('MyModel');
add_block('simulink/Math Operations/Gain', 'MyModel/G', ...
    'Gain','2', 'Position',[100 100 140 130]);
add_line('MyModel','G/1','Scope/1','autorouting','on');
set_param('MyModel','StopTime','10');
```

- 배치가 흐트러졌으면

```
tidy_layout('SB9_continuous_done')
```

L-5. 해 볼 것 (`SB12_reuse_todo`)

1. 1차 지연을 만들고 **Create Subsystem from Selection**
2. **Create Mask** 로 `tau` 파라미터 추가, 안의 `Gain` 을 `1/tau` 로
3. 복사해서 두 개로 만들고 `tau` 를 0.1 / 1.0 로
4. Scope 로 겹쳐 보고 63 % 도달 시간이 각각 `tau` 인지 확인
5. Icon 탭에 식을 표시해 보기
6. `build_w06_1_models.m` 을 열어 `makeLagSubsystem` 함수를 읽는다.
방금 손으로 한 일을 코드가 어떻게 하는지 대조할 것

마무리

이번 주차 요약

순서	한 일	확인 방법
1	블록 놓고 이어서 실행	Scope 에 사인파
2	솔버를 고정 스텝 0.05 로	Ctrl + E 화면
3	MATLAB Function 으로 계산	WrapPi 가 -2
4	Subsystem 으로 묶기	Calc 포트가 a , b , y
5	Goto/From 으로 선 없이 잇기	Display 에 12
6	버스 만들고 꺼내고 바꾸기	Inner.y =2, Inner.z =99
7	PID · 포화 · 안티와인드업	지연이 19.8초 → 2.6초
8	변수 · 로깅 · Data Inspector	out.logout 표본 201개
9	Mux·Demux·Selector 로 묶고 풀기	Selector 가 2 출력
10	Unit Delay 누적기	10초 뒤 10.05
11	Integrator·Transfer Fcn	63 % 도달이 0.300 s
12	Switch·Multiport Switch·Stop	60초 설정인데 15초에 정지
13	Enabled vs Triggered	201 대 5
14	마스크 씌우기	tau 0.1 / 1.0 두 곡선

수업 진도 체크

★ 6주차 수업을 따라가기 위한 최소 조건

1일차 이론 이해 (A~F)

- ☐ 블록 · 포트 · 신호선 · 서브시스템 · 버스 · 솔버를 각각 설명할 수 있다
- ☐ 본 과목이 고정 스텝 · 이산 · 0.05초를 쓰는 이유를 말할 수 있다
- ☐ MATLAB Function 블록의 포트가 함수 첫 줄을 따라간다는 것을 안다
- ☐ 중첩된 버스에서 전체 경로를 써야 하는 이유를 안다
- ☐ Goto/From 을 쓰면 좋은 점과 나쁜 점을 각각 말할 수 있다
- ☐ 적분 와인드업이 왜 생기는지 설명할 수 있다

1일차 실습 완료 (A~F)

- ☐ SB1_first_todo — 솔버를 고정 스텝으로 고치고 Scope 확인
- ☐ SB2_mfcn_todo — 세 함수를 고쳐 12 / 14 , 6 / -2 출력
- ☐ SB2 — 출력 미할당 오류를 일부러 내 보고 메시지 확인
- ☐ SB3_subsys_todo — Ctrl + G 로 묶고 포트 이름 바꾸기
- ☐ SB3 — Goto/From 으로 이어 12 출력
- ☐ SB4_bus_todo — Inner.y 뽑아 2 출력
- ☐ SB4 — Bus Assignment 로 Inner.z 를 99 로

- ☐ `SB5_pid_todo` — P → PI → 포화 → 안티와인드업 순서로 4번 실행
- ☐ `SB6_param_todo` — 변수화, 로깅, Data Inspector 겹쳐 보기

2일차 이론 이해 (G~L)

- ☐ 벡터와 버스의 차이를 표로 설명할 수 있다
- ☐ `Unit Delay` 가 **대수 루프를 끊는 원리**를 설명할 수 있다
- ☐ 1차 지연의 `tau` 가 **63.2 % 도달 시간**임을 안다
- ☐ `Multiport Switch` 로 모드를 고를 때 **둘 다 계산하는 이유**를 말할 수 있다
- ☐ `Enabled` 와 `Triggered` 의 차이를 실측 숫자로 설명할 수 있다
- ☐ 모델을 **코드로 생성**하는 세 가지 이유를 말할 수 있다

2일차 실습 완료 (G~L)

- ☐ `SB7_signal_todo` — Mux·Demux·Selector 로 **2** 뽑기
- ☐ `SB8_discrete_todo` — 누적기 10초 뒤 **10.05**
- ☐ `SB8` — Unit Delay 를 빼고 **대수 루프 오류를 직접 봄**
- ☐ `SB9_continuous_todo` — 63 % 도달 **0.300 s** 확인
- ☐ `SB9` — 적분기 출력 제한으로 **3** 에서 멈추는 것 확인
- ☐ `SB10_logic_todo` — 60초 설정인데 15초에 멈추는 것 확인
- ☐ `SB11_enabled_todo` — Enabled **201** 대 Triggered **5** 확인
- ☐ `SB12_reuse_todo` — 마스크를 만들고 tau 두 값으로 비교

관찰 기록

- ☐ P 제어만일 때 정상상태 오차가 얼마였는지 적어 두었다
- ☐ 안티와인드업 on/off 의 차이를 초 단위로 적어 두었다
- ☐ 변수가 없을 때 나는 오류 메시지를 적어 두었다

과제 W06-0A — 1일차 여섯 모델과 측정

- **제출 기한:** 6주차 수업 전
- **제출:** 완성한 `.slx` 6개 + 스크린샷 + 수치 + 짧은 분석

① 여섯 개 빈칸본 완성

- `SB1` ~ `SB6` 의 `_todo` 를 모두 채운다
- 각 모델의 **Scope** 또는 **Display** 스크린샷을 첨부한다

② 검증 — 필수

★ 결과는 수치로 제시한다

(a) `WrapPi` 가 맞게 동작하는가

- `ang_a` , `ang_b` 를 아래 세 조합으로 바꿔 결과를 표로 제출

<code>ang_a</code>	<code>ang_b</code>	단순 차이	<code>WrapPi</code> 결과
179°	-179°	358°	
10°	350°		
45°	-45°		

- 세 번째 줄은 **wrap** 이 필요 없는 경우다. 그대로 나오는지 확인하는 것이 목적이다
- 블록에 각도를 넣을 때는 $179 \cdot \pi / 180$ 처럼 **라디안**으로 적는다

(b) 안티와인드업 on/off 비교

- SB5 에서 **none** 과 **clamping** 을 각각 실행
- **Data Inspector** 로 두 Run 을 겹친 화면을 캡처해서 제출
- 커서로 읽은 수치를 표로 제출

안티와인드업	목표 0.5 에 닿은 시각 [s]	10초 이후 지연 [s]
none		
clamping		

(c) 로깅 결과

- SB6 에서 **K_gain** 을 2 와 5 로 두 번 실행
- **SB_plot(out)** 이 출력한 표본 개수 · 최댓값을 제출

③ 분석 (5~10줄)

1. P 제어만 썼을 때 목표 2 에서 출력이 1.0 에 멈춘 이유는 무엇인가
2. 적분기를 넣었더니 왜 목표를 맞추게 되었는가
3. 안티와인드업을 끄면 왜 10초 이후에도 계속 밀어 대는가
4. 블록에 숫자 대신 변수 이름을 쓰는 방식의 단점은 무엇인가

평가 기준

항목	배점
여섯 모델 완성 및 정상 동작	30%
WrapPi 검증 표 (수치 정확성)	20%
안티와인드업 비교 수치 + Data Inspector 캡처	25%
로깅 결과 수치	10%
분석의 타당성	15%

검증 항목 합계 55% — 측정하지-않은-성공은-성공이-아니다

과제 W06-0B — 2일차 여섯 모델과 측정

- **제출 기한**: 7주차 수업 전
- **제출**: 완성한 **.slx** 6개 + 스크린샷 + 수치 + 짧은 분석

① 여섯 개 빈칸본 완성

- SB7 ~ SB12 의 **_todo** 를 모두 채운다
- 각 모델의 **Scope** 또는 **Display** 스크린샷을 첨부한다

② 검증 — 필수

★ 아래 숫자는 기준 환경 실측값값이다. 학습자 것과 맞아야 한다

- (a) 누적기와 기성 블록이 같은가 (SB8)

방식	10초 뒤 값
Unit Delay 누적기	
Discrete-Time Integrator (Backward Euler)	
같은 블록 (Forward Euler)	

- 세 값을 채우고, **Forward** 와 **Backward** 가 왜 다른지 한 줄로 설명

(b) 대수 루프를 직접 만들어 보기 (SB8)

- Unit Delay 를 빼고 실행 → 오류 메시지 전문을 그대로 붙일 것
- Debug → Diagnostics → Algebraic Loops → Highlight 화면 캡처

(c) 시상수 측정 (SB9 , SB12)

블록	설정 tau	63 % 도달 시간 [s]
Transfer Fcn	0.3	
Lag_fast	0.1	
Lag_slow	1.0	

- 커서로 읽은 값을 적을 것. 설정값과 몇 % 차이인가

(d) 조건부 실행 비교 (SB11)

서브시스템	20초 뒤 누적값	Output when disabled = reset 일 때
Enabled		
Triggered		

(e) 솔버 바꿔 보기 (SB9)

- ode45 ↔ 고정 스텝 0.05 로 각각 실행
- 두 곡선의 최대 차이를 수치로 제출
- tau = 0.05 로 줄이고 고정 스텝 0.05 로 돌린 결과도 함께

③ 분석 (5~10줄)

- 벡터와 버스를 각각 어디에 써야 하는가. 바꿔 쓰면 무엇이 곤란해지는가
- Unit Delay 가 대수 루프를 끊는다는 것은 물리적으로 무슨 뜻인가
- Enabled 를 ROS IsNew 에 붙이면 무엇이 좋아지는가
- 시상수보다 큰 고정 스텝을 쓰면 왜 결과가 부정확해지는가
- 모델을 손으로 그리지 않고 코드로 만드는 방식의 단점은 무엇인가

평가 기준

항목	배점
여섯 모델 완성 및 정상 동작	25%
(a) 누적기 세 값 + 이유	15%
(b) 대수 루프 오류 재현 + 캡처	15%
(c) 시상수 측정 표	15%
(d) 조건부 실행 비교 표	10%
(e) 솔버 비교 수치	5%
분석의 타당성	15%

검증 항목 합계 60% — 측정하지-않은-성공은-성공이-아니다

막혔을 때

증상	원인	해결
선이 안 그려짐	출력→입력 방향이 아님	오른쪽 화살표에서 시작해 왼쪽 화살표로
실행이 순식간에 끝남	정지 시간이 짧음	툴스트립의 정지 시간 확인
스텝 간격이 들쭉날쭉	가변 스텝 솔버	<code>Ctrl + E</code> → Fixed-step / discrete / 0.05
함수 블록 포트가 안 늘어남	저장을 안 함	코드 편집기에서 <code>Ctrl + S</code>
<code>Output argument ... not assigned</code>	<code>if</code> 분기에서 출력 미정의	모든 분기에서 출력을 정한다
배열 크기 오류	루프에서 배열이 자람	<code>zeros()</code> 로 미리 크기 확보
Bus Selector 에서 신호를 못 찾음	경로가 짧음	<code>y</code> 가 아니라 <code>Inner.y</code> 로 전체 경로
버스 원소 이름이 <code>signal1</code>	신호선에 이름이 없음	선을 우클릭 → Properties → 이름 지정
<code>From</code> 이 값을 못 받음	태그가 다름	<code>Goto</code> 와 <code>From</code> 의 태그 철자 확인
<code>파라미터 ...에 대한 설정이 유효하지 않습니다</code>	변수가 작업공간에 없음	<code>SB_setup</code> 먼저 실행
<code>out.logout</code> 이 비어 있음	로깅을 안 컴	신호선 우클릭 → Log Selected Signals
모델이 깨짐	—	<code>build_w06_0_models</code> 다시 실행

모델을 다시 만들어야 할 때

```
cd('<배포 폴더>/W06_0_simulink')
build_w06_0_models      % 1일차 SB1~SB6
build_w06_1_models      % 2일차 SB7~SB12
```

- 해당 모델이 모두 새로 만들어진다. `_todo` 에 하던 작업은 사라진다

- 배치만 흐트러진 것이라면 다시 만들 필요 없다

```
tidy_layout('SB9_continuous_done')
```

- 스크립트를 읽어 보면 **각 블록이 어떻게 만들어지는지** 코드로 알 수 있다

참고 자료

공식 문서

- Simulink 시작하기 — <https://www.mathworks.com/help/simulink/getting-started-with-simulink.html>
- MATLAB Function 블록 — <https://www.mathworks.com/help/simulink/slref/matlabfunction.html>
- 서브시스템 만들기 — <https://www.mathworks.com/help/simulink/ug/create-a-subsystem.html>
- 버스 다루기 — <https://www.mathworks.com/help/simulink/ug/composite-signals.html>
- PID Controller 블록 — <https://www.mathworks.com/help/simulink/slref/pidcontroller.html>
- Simulation Data Inspector — <https://www.mathworks.com/help/simulink/ug/simulation-data-inspector-overview.html>
- Unit Delay 와 이산 시스템 — <https://www.mathworks.com/help/simulink/ug/discrete-and-continuous-systems.html>
- Integrator 블록 — <https://www.mathworks.com/help/simulink/slref/integrator.html>
- 조건부 실행 서브시스템 — <https://www.mathworks.com/help/simulink/ug/conditionally-executed-subsystems-overview.html>
- 마스크 만들기 — <https://www.mathworks.com/help/simulink/ug/create-a-simple-mask.html>
- 솔버 고르기 — <https://www.mathworks.com/help/simulink/ug/choose-a-solver.html>
- 프로그램으로 모델 만들기 — <https://www.mathworks.com/help/simulink/programmatic-modeling.html>

교재

- **Simulink Fundamentals.pdf** (볼트 루트) — MathWorks 공식 교육 교재 333쪽
 - 이 문서의 A~J 절은 이 교재의 **29장 순서를 그대로 따른다**

이 문서	교재 장
A	2. Creating and Simulating a Model
B, J	3. Modeling Programming Constructs
H	4. Modeling Discrete Systems
I	5. Modeling Continuous Systems
C, D, G, L	6. Developing Model Hierarchy
K	7. Modeling Conditionally Executed Algorithms
L	8. Referencing Model Components · 9. Creating Libraries
E, F	부록 A. Debugging · F. Solver Selection

볼트 내 문서

- [W06_Simulink_ROS2_연동과_첫_제어기](#) — 이번 주차에 학습한 내용 위에 ROS 2 를 엮는다
- [측정하지-않은-성공은-성공이-아니다](#) — 과제 배점의 근거

배포 파일

파일	내용
W06_0_simulink/SB1_first_todo.slx · _done.slx	A. 첫 모델과 솔버
W06_0_simulink/SB2_mfcn_todo.slx · _done.slx	B. MATLAB Function
W06_0_simulink/SB3_subsys_todo.slx · _done.slx	C. Subsystem · Goto/From
W06_0_simulink/SB4_bus_todo.slx · _done.slx	D. 버스
W06_0_simulink/SB5_pid_todo.slx · _done.slx	E. PID · 포화 · 안티와인드업
W06_0_simulink/SB6_param_todo.slx · _done.slx	F. 파라미터 · 로깅
W06_0_simulink/SB_setup.m	실습 F 파라미터
W06_0_simulink/SB_plot.m	실습 F 결과 그래프
W06_0_simulink/SB7_signal_todo.slx · _done.slx	G. Mux · Demux · Selector
W06_0_simulink/SB8_discrete_todo.slx · _done.slx	H. 샘플타임 · Unit Delay · Rate Transition
W06_0_simulink/SB9_continuous_todo.slx · _done.slx	I. Integrator · Transfer Fcn · 솔버
W06_0_simulink/SB10_logic_todo.slx · _done.slx	J. Switch · Multiport Switch · Stop
W06_0_simulink/SB11_enabled_todo.slx · _done.slx	K. Enabled · Triggered 서브시스템
W06_0_simulink/SB12_reuse_todo.slx · _done.slx	L. 마스크
W06_0_simulink/build_w06_0_models.m	1일차 12개 모델 생성
W06_0_simulink/build_w06_1_models.m	2일차 12개 모델 생성
W06_0_simulink/tidy_layout.m	배치·색 복구

다음 예고

- 6주차 — Simulink ROS 2 연동과 첫 제어기
- 이번 주차에 학습한 블록 위에 **ROS 2 블록 세 개**가 얹힌다
 - **Subscribe** — 토픽을 받아 **버스**로 내보냄 → 이번 주차 D절
 - **Blank Message** + **Bus Assignment** — 메시지 채우기 → 이번 주차 D절
 - **Publish** — 토픽으로 발행
- 2일차 G~L 절의 블록도 곧바로 쓰인다
 - **Unit Delay** — 웨이포인트 번호 되먹임 (7주), 대수 루프 차단 (9주)
 - **Transfer Fcn** — 모터 1차 지연 (7주 **Thrusters**)
 - **Multiport Switch** — 유도법칙 갈아 끼우기 (9주 **ModeSwitch**)
 - **Stop Simulation** — 임무 종료 (8-9주)
- 준비물
 - 두 과제(W06-0A, W06-0B)를 끝내 둘 것. 특히 **버스·MATLAB Function·Unit Delay**